

AUTHENTICATION & AUTHORIZATION

COMS 4156 ADVANCED SWE

21st November, 2024

Our Team

Abhishek Paul

ap4623



@abhishekpaul11

Aditi Chowdhuri

anc2207



@Aditi-Chowdhuri

Rahaf Bin Muhanna

rb3713



@rmuhanna

Puja Singla

ps3467



@pujasingla

Garlic Bread

What to expect?

Introduction

Session Tokens

OAuth 2.0

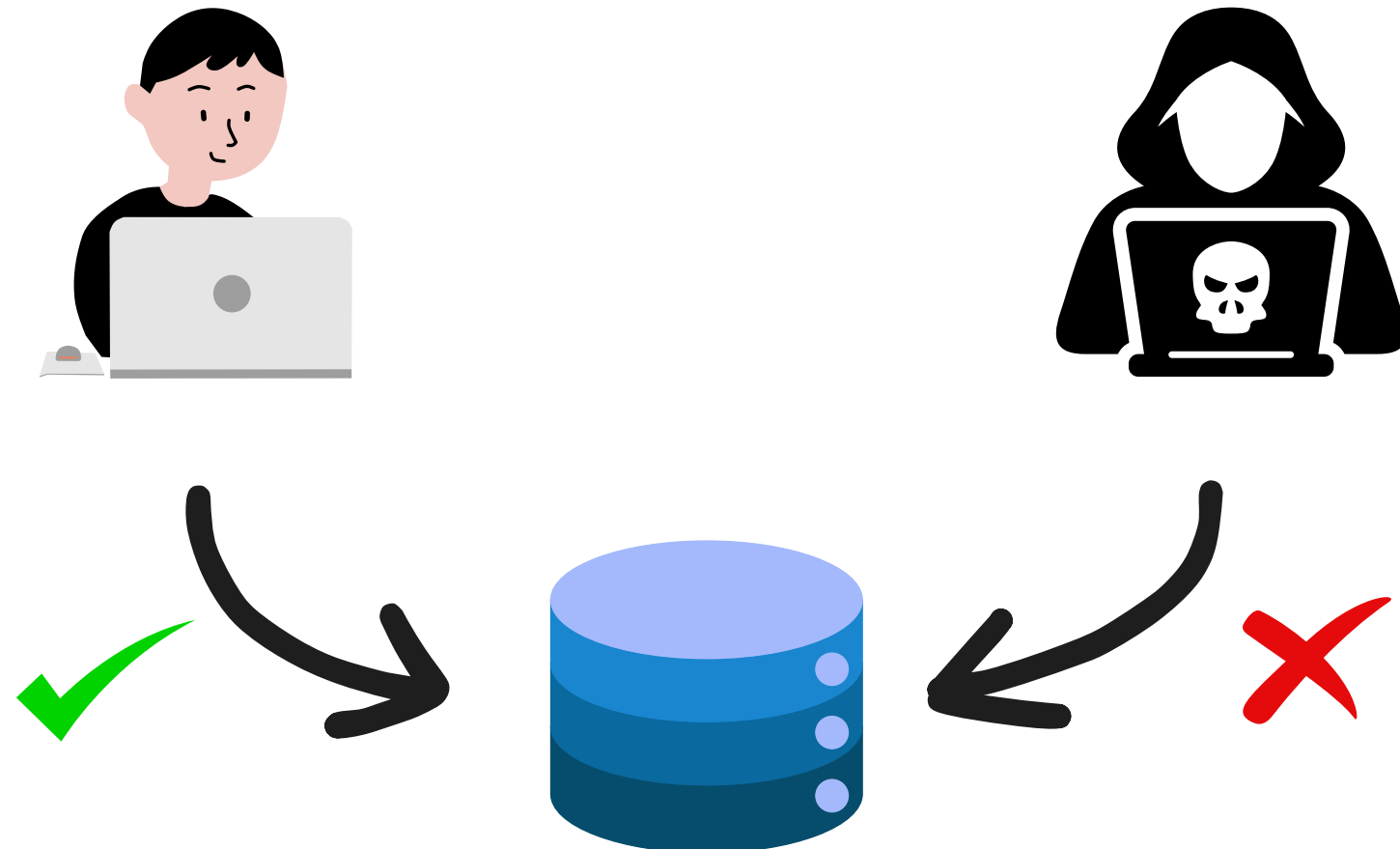
OpenID Connect Tokens (JWT)

Demo



Authentication

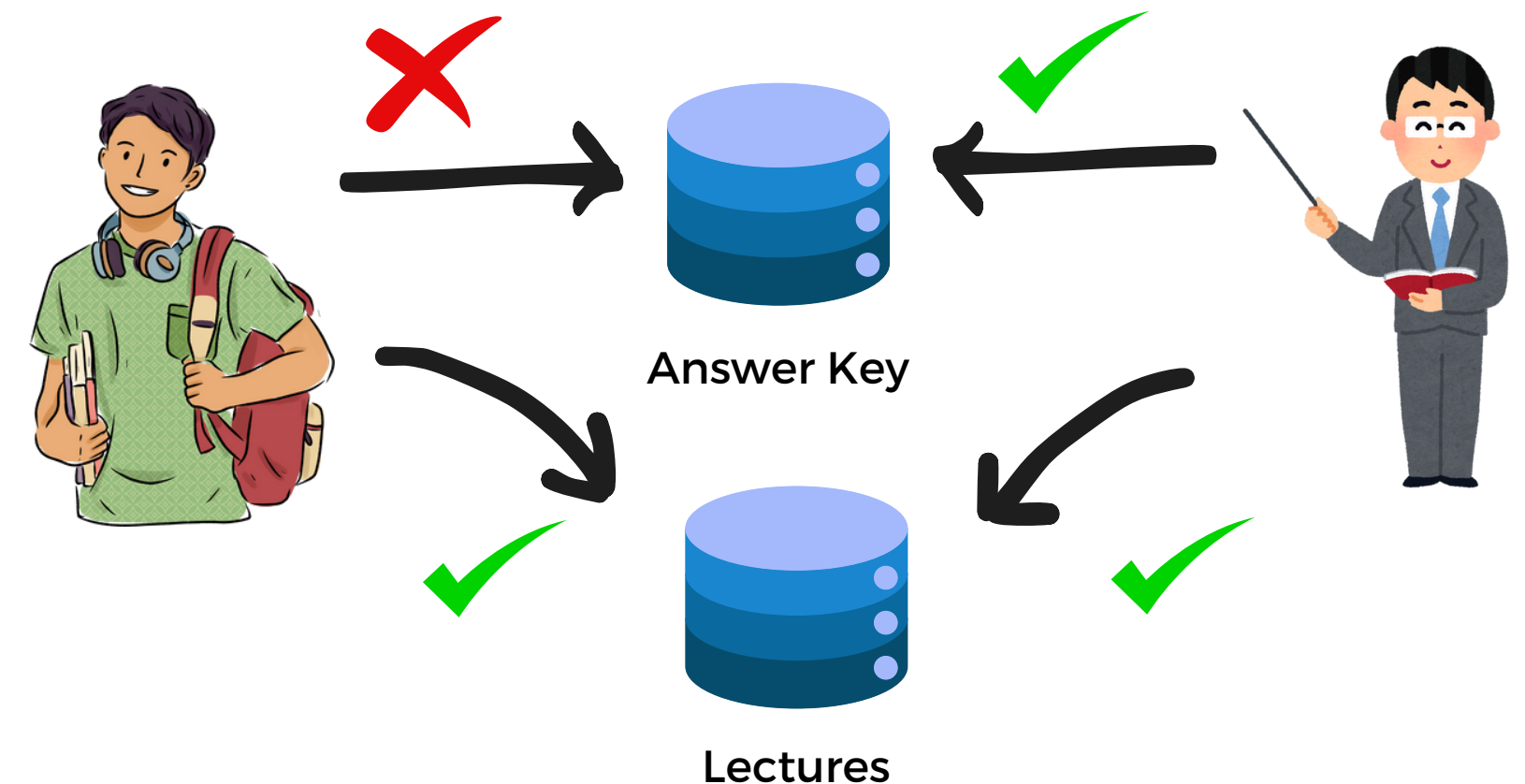
- Ensures only legitimate users have access to resources.
- Prevents data leaks.



What are they? Why do we need them?

Authorization

- Ensures specific type of users have access to specific type of resources.
- Prevents sensitive information from being available to everyone.

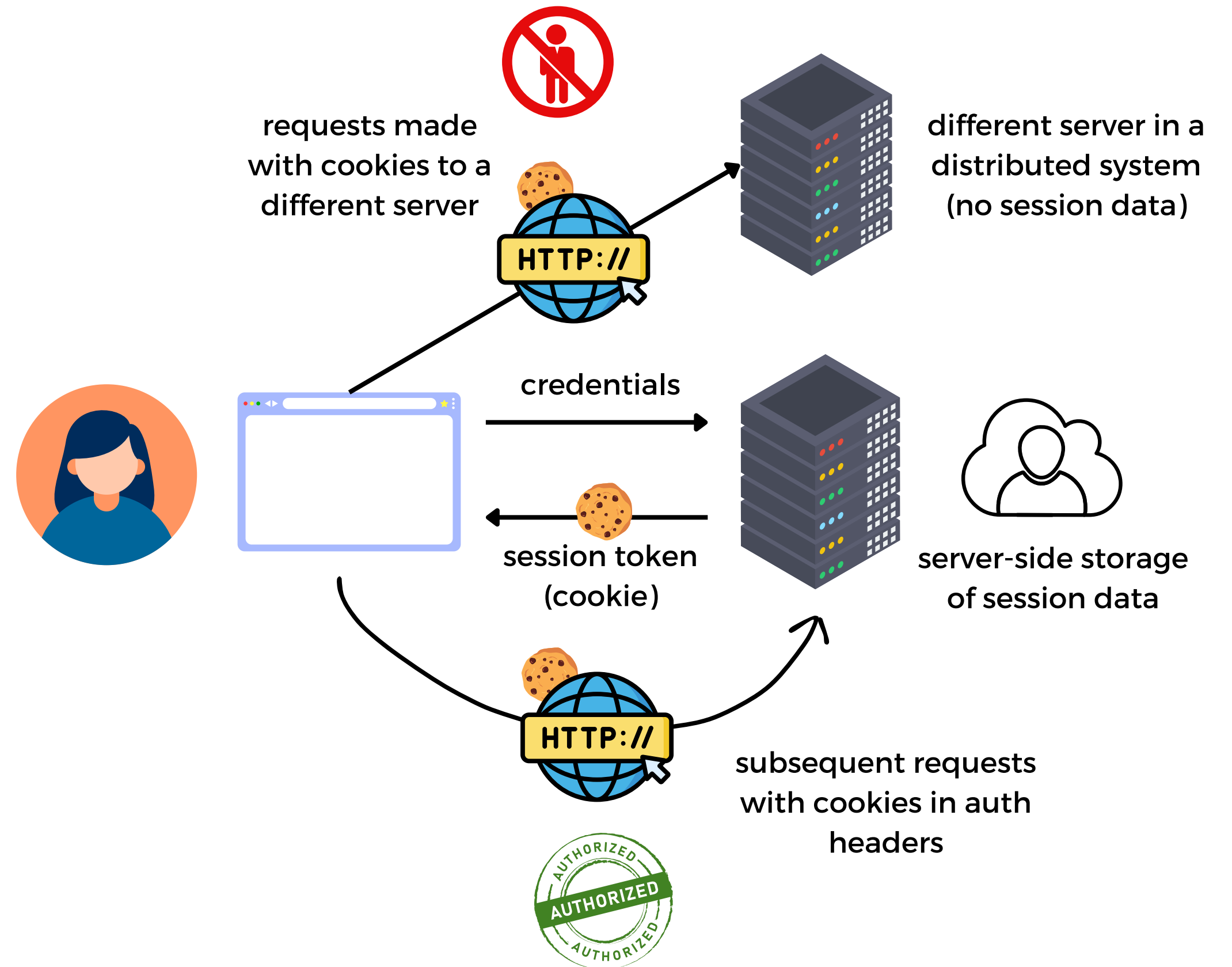


Garlic Bread

- **Authentication focused:** Cannot handle additional scopes or roles (no authorization).
- **Stateful:** Relies on the server side storage of session data.
- **Scalability issues:** Ideally does not work with multiple servers.
- **Short lived:** Tokens expire when user logs out or closes the browser.

Used in traditional web apps with client-server model

Session Tokens

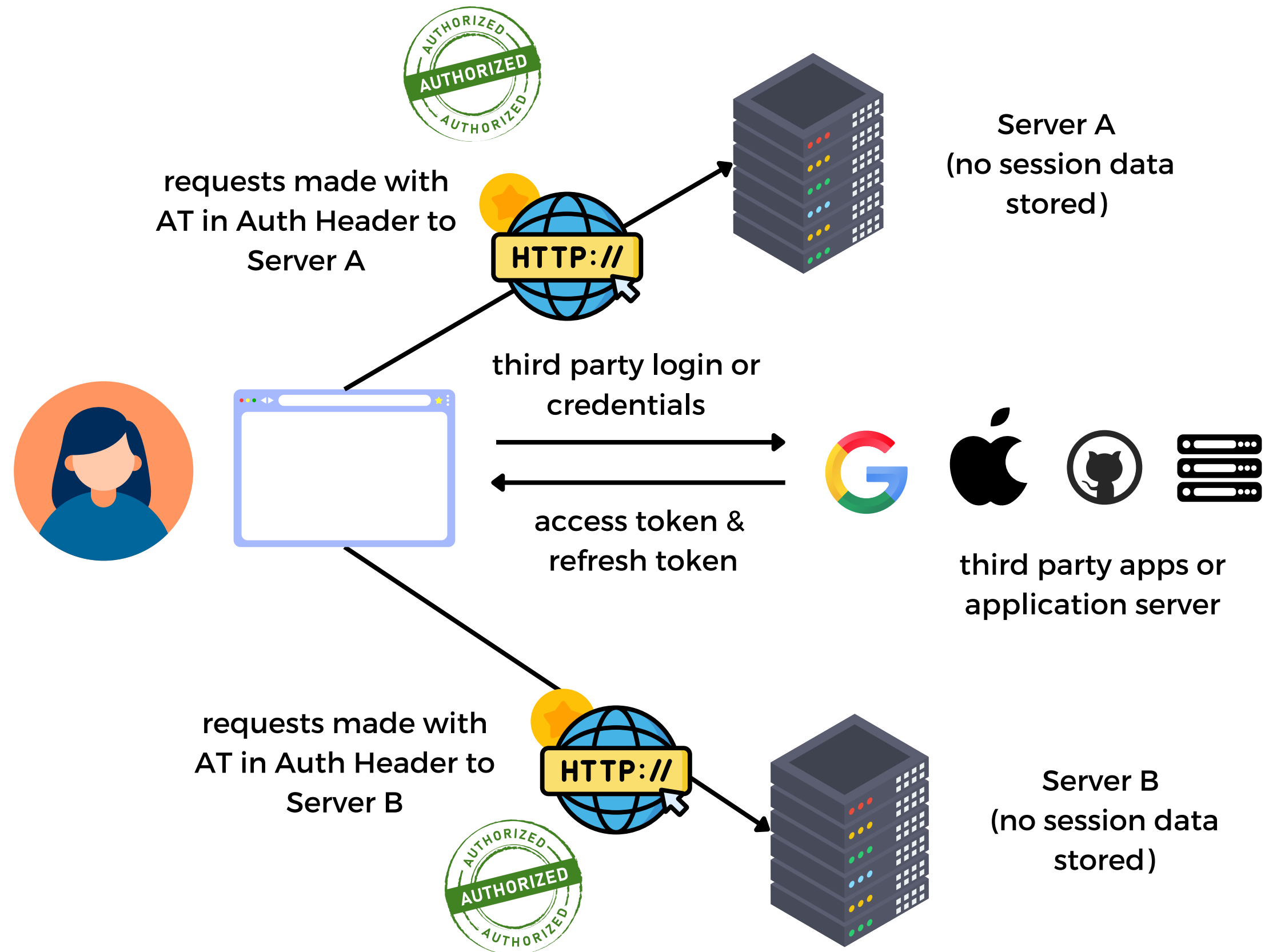


Garlic Bread

- **Authorization supported:** Tokens have provisions to handle scopes or roles.
- **Stateless:** Tokens are structured (store user data), so no need of server side storage. Works with distributed systems.
- **Supports delegation:** We can use third party sign-ins.
- **Refreshable:** Access Tokens have an expiry time, but Refresh Tokens can be used to get a new Access Token without another login.

Used in mobile apps, microservices and modern APIs

OAuth 2.0



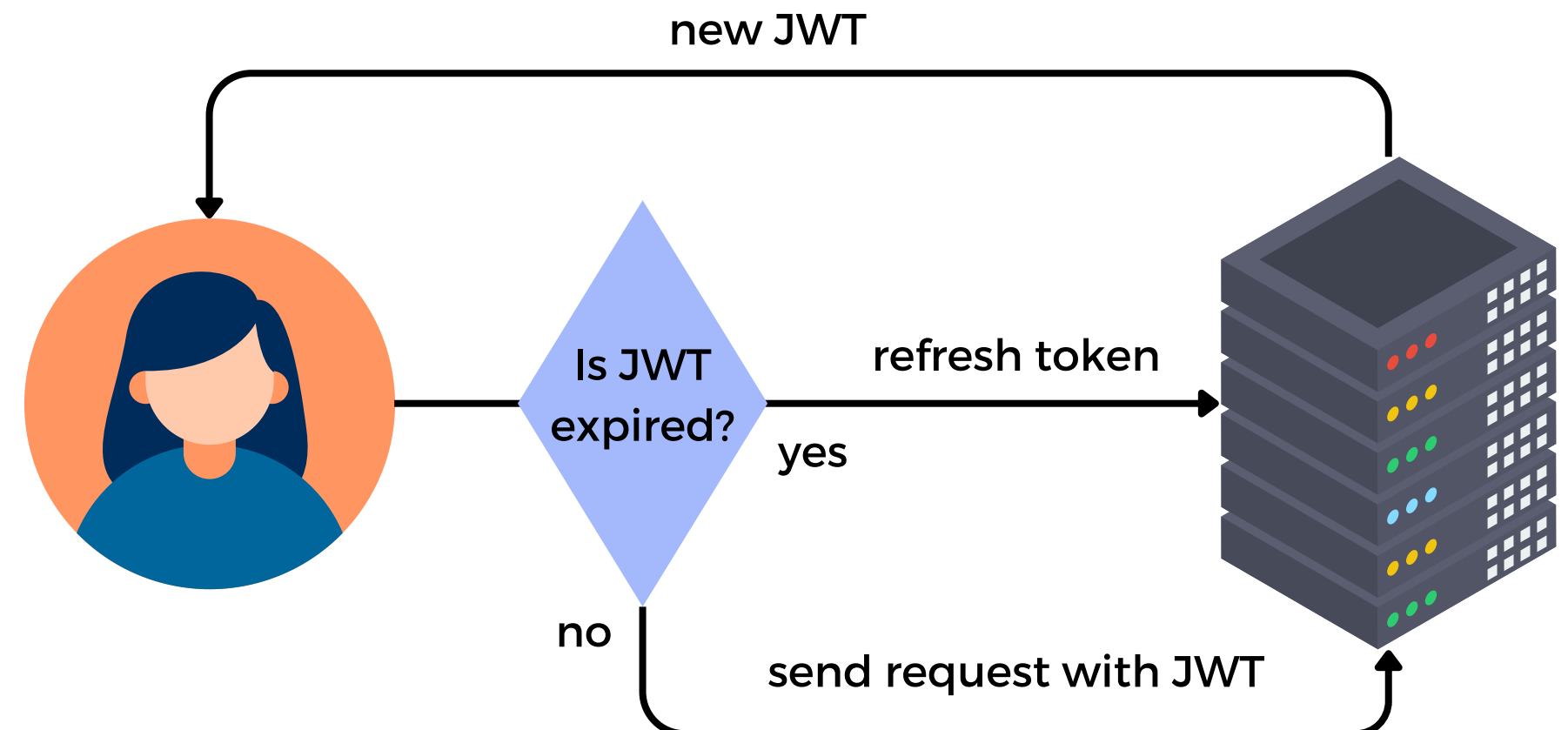
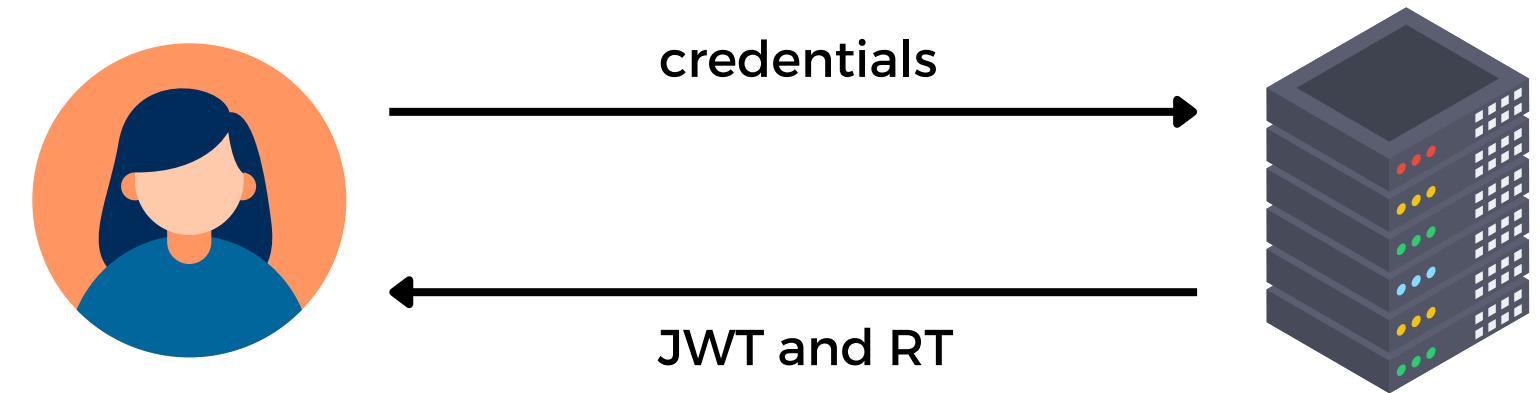
Garlic Bread

- Json Web Tokens (JWT) is **not part of the OAuth 2.0 framework**.
- Commonly used token type for authentication (OpenID Connect or OIDC layer above OAuth 2.0).
- Upon verification of credentials, the server issues an access token and a refresh token.
- **Access Token:** JWT (structured)
- **Refresh Token:** not JWT (alphanumeric gibberish)
- Signature of the JWT in request headers is verified by the server before proceeding.

Limitations

- Large size (as its structured)
- Cannot revoke a leaked JWT - have to wait till its expired :(

OIDC Tokens (JWT)



JWT (or OIDC tokens) consists of three parts (base64Url-encoded) separated by a dot.

Header.Payload.Signature

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

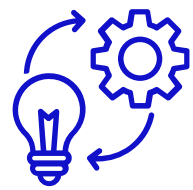
- Algorithm Used for signing
- Token Type

```
{  
  "sub": "1234567890",  
  "iss": "garlicbread.com",  
  "role": ["admin"],  
  "exp": 1516240022  
  "iat": 1516239022  
}
```

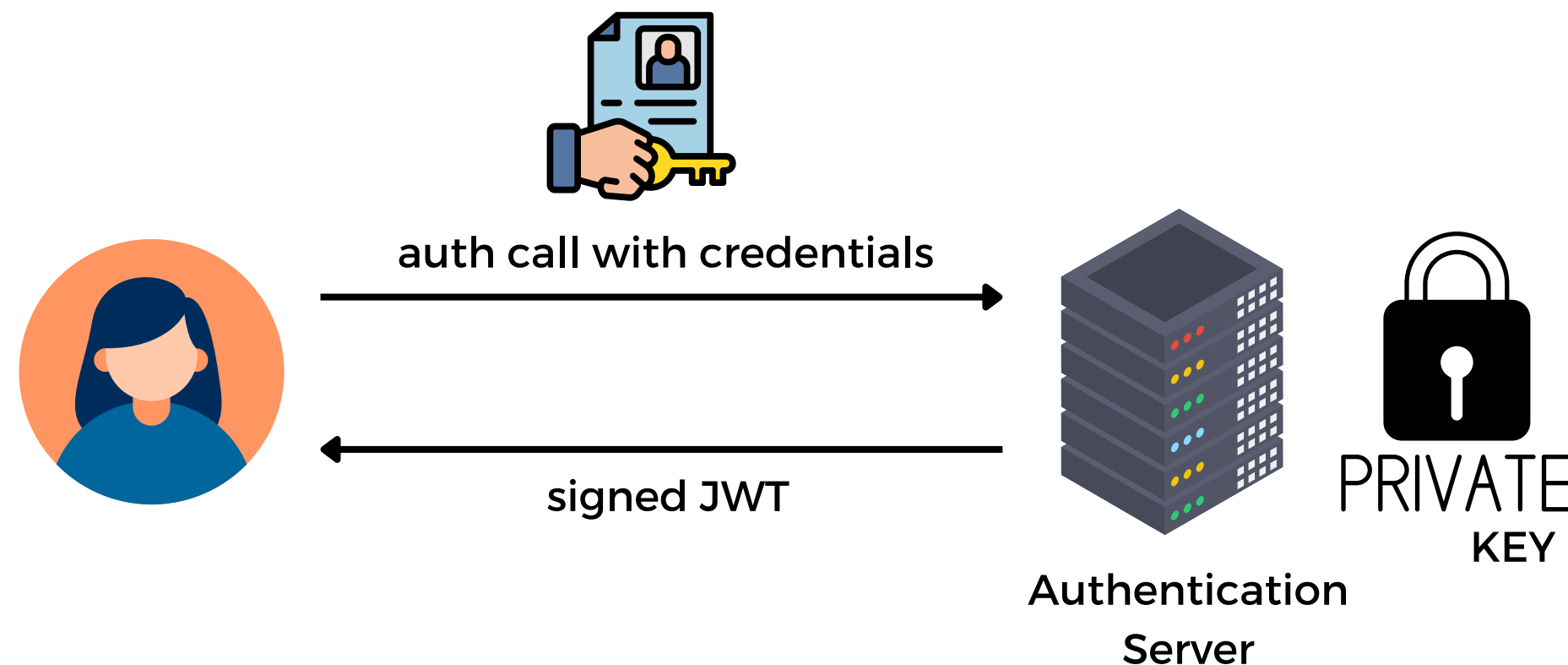
- Registered Claims
 - sub (subject)
 - iss (issuer)
 - iat (issued at)
 - exp (expiry)
- Public Claims
 - roles
 - permissions
- Private Claims
 - company specific fields

```
HMACSHA256(  
  base64UrlEncode(header) + "." + base64UrlEncode(payload),  
  secret  
)
```

- Signing Key used for verification
- Secret for symmetric algorithms (HS256)
- Private key for asymmetric keypair algorithms (RS256)
- This key is needed to make some changes in the payload



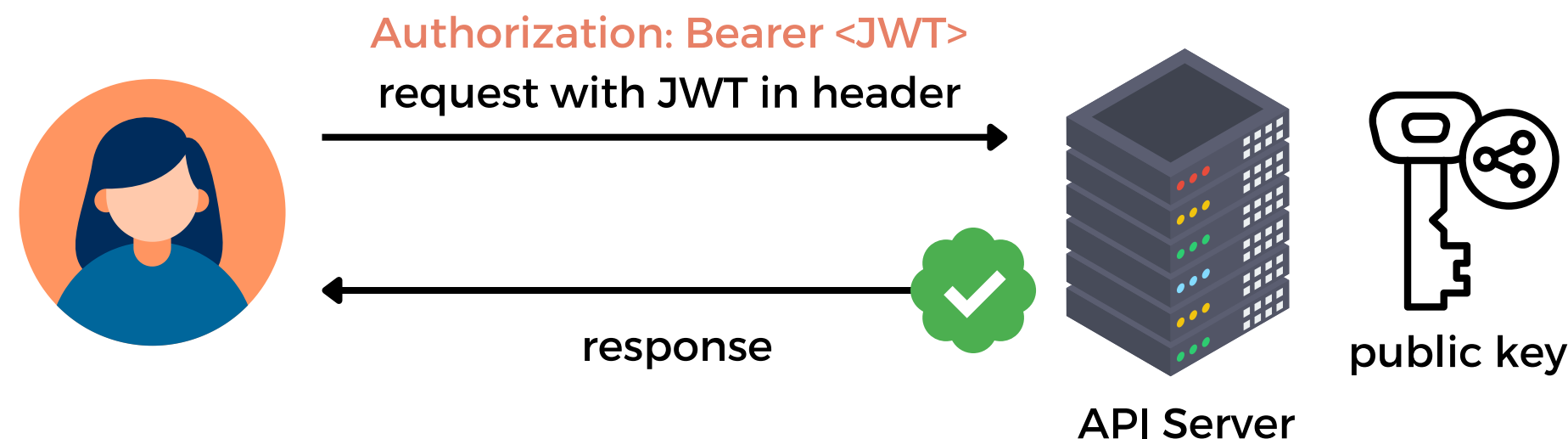
JWT Issuance



- generates JWT with header and payload
- signs it with the secret (symmetric algorithm) or private key (asymmetric algorithm)



JWT Verification



- verifies the JWT signature with the public key and checks if its legitimate (not tampered with)
- checks if its expired

DEMO TIME !!!

J W T A U T H

VIEW DEMO



THANK YOU

ANY QUESTIONS?